

Módulo 1: Fundamentos y Entorno

T02: Docker y Variables de Entorno

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *Contenedores, .env y el principio de no hardcodear secretos*

Objetivos

- Entender qué es Docker y por qué mejora la reproducibilidad del entorno
- Distinguir entre imagen, contenedor y volumen
- Gestionar configuración con variables de entorno y archivos `.env`
- Identificar por qué los secretos hardcodeados son una vulnerabilidad crítica

¿Por qué Docker en un curso de seguridad?

- **Reproducibilidad** — todos con el mismo entorno
- **Aislamiento** — la app vulnerable no toca tu sistema
- **Desechable** — si rompes algo,
`docker compose down && up`
- **Refleja producción** — así se despliegan apps reales

📦 Conceptos clave:

- **Imagen** → plantilla read-only
- **Contenedor** → instancia en ejecución
- **Volumen** → persistencia de datos
- **Docker Compose** → orquestar varios contenedores

Variables de entorno — la forma correcta de configurar

```
# .env (NUNCA se sube a Git)
JWT_SECRET=s3cr3t-k3y-muy-largo-y-aleatorio
DB_PATH=./vuln.db
PORT=3000
```

```
// En el código - lee de process.env
const JWT_SECRET = process.env.JWT_SECRET || 'fallback-inseguro';
```

💀 **VulnApp hace esto MAL:** `const JWT_SECRET = 'secret';` → hardcodeado en `src/routes/auth.ts:7`

El peligro de hardcodear secretos

```
// ⚠ VULNERABLE - src/routes/auth.ts línea 7  
const JWT_SECRET = 'secret';
```

Consecuencias:

1. Cualquiera con acceso al repo puede firmar tokens
2. El secreto aparece en el historial de Git para siempre
3. No se puede rotar sin redesplegar
4. `git log -p --all -S 'secret'` → revelado

Fix:

```
const JWT_SECRET = process.env.JWT_SECRET!;  
// + validar al arrancar que no esté vacío
```

`.env` y `.gitignore` — regla de oro

```
# .gitignore
.env
.env.*
!.env.example
```

```
# .env.example (Sí se sube - sin valores reales)
JWT_SECRET=
DB_PATH=./vuln.db
PORT=3000
```

✓ Buena práctica: `.env.example` documenta qué variables necesita la app, sin revelar valores.

Docker Compose – VulnApp en un comando

```
# docker-compose.yml
services:
  vulnapp:
    build: .
    ports:
      - "3000:3000"
    env_file:
      - .env
    volumes:
      - ./vuln.db:/app/vuln.db
```

```
docker compose up --build    # arranca todo
docker compose down -v      # destruye todo (incluido volumen)
```

Dockerfile — buenas prácticas de seguridad

```
# ✅ Imagen base mínima y versionada
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev      # solo dependencias de producción

FROM node:20-alpine
# ✅ Usuario sin privilegios
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY --from=build /app/node_modules ./node_modules
COPY . .
USER app                  # ← NUNCA ejecutar como root
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

- ✅ Multi-stage build → imagen final sin herramientas de compilación (menor superficie de ataque).

Docker networking – aislamiento de red

```
# docker-compose.yml con redes aisladas
services:
  vulnapp:
    networks: [frontend, backend]
  database:
    networks: [backend]      # ← NO expuesta al exterior

networks:
  frontend:
  backend:
    internal: true          # sin acceso a internet
```

Red	Acceso externo	Contenedores
frontend	Sí (puertos mapeados)	vulnapp
backend	No (internal: true)	vulnapp, database

💡 La base de datos nunca debería ser accesible directamente desde fuera del cluster.

Validación de configuración al arrancar

```
// src/config.ts - patrón "fail fast"
function requireEnv(name: string): string {
  const value = process.env[name];
  if (!value) {
    console.error(`❌ Variable ${name} no definida. Abortando.`);
    process.exit(1);
  }
  return value;
}

export const config = {
  JWT_SECRET: requireEnv('JWT_SECRET'),
  DB_PATH: process.env.DB_PATH || './vuln.db',
  PORT: parseInt(process.env.PORT || '3000', 10),
};
```

⚠️ Nunca uses fallbacks para secretos: `process.env.JWT_SECRET || 'secret'` → el fallback se vuelve la clave en producción.

Herramientas para detectar secretos en repos

Herramienta	Qué hace
git-secrets	Pre-commit hook que bloquea patrones
truffleHog	Escanea historial de Git buscando alta entropía
gitleaks	SAST para secretos en CI/CD
GitHub Secret Scanning	Alerta automática en repos públicos

⚠ Si ya subiste un secreto, rotarlo inmediatamente. Borrar el commit NO es suficiente (forks, caches).

Lab T02: Secretos en el código

Objetivo: detectar secretos hardcodeados en VulnApp y moverlos a variables de entorno

Setup: `cd VulnApp && code .`

1. Busca en `src/` cualquier valor hardcodeado (clave JWT, contraseñas, connection strings)
2. Crea un archivo `.env` con esos valores
3. Actualiza el código para leer de `process.env`
4. Verifica que la app sigue funcionando con `npm start`